

MI GFX API

1 Overview

1.1. Module Description

GFX (Graphic Engine) hardware provides fast graphics rendering function for user-painted UI.

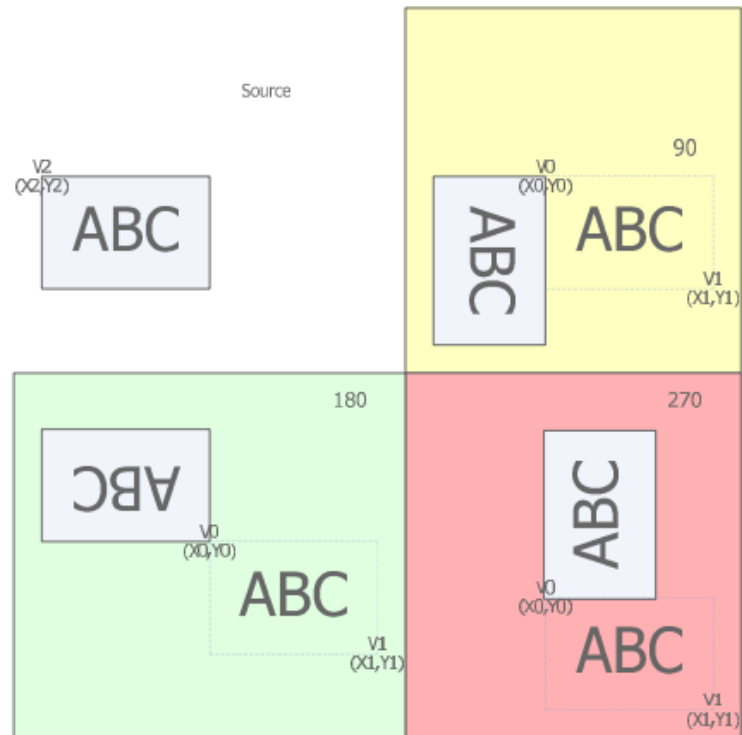
There are:

- rectangle color fill
- Bitmap moving (support scaling, rotation, mirror flip, format conversion, alpha blending overlay, ColorKey, etc.)

Immediately 仅 Tauyaki, Takoyaki, Ikayaki series core piece support GFX.

Rotate: HW supports 90/180/270 degree rotation

Rotate



Picture 1-1

Alpha Blend Overlay: supports normal 2D alpha manipulation

Alpha Blending

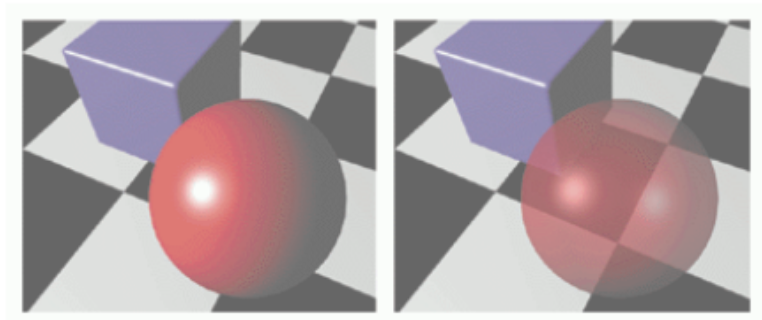


Figure 1-2

Mirroring: Support H mirroring, V mirroring, HV mirroring at the same time

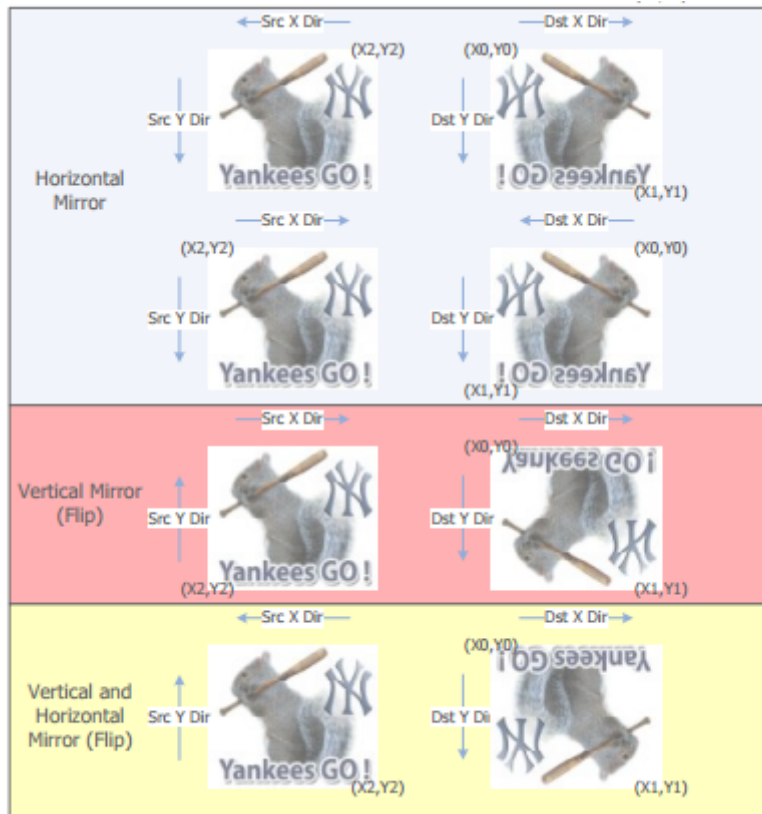


Figure 1-3

Scaling: support scaling during bitblt

Stretch Bitblt

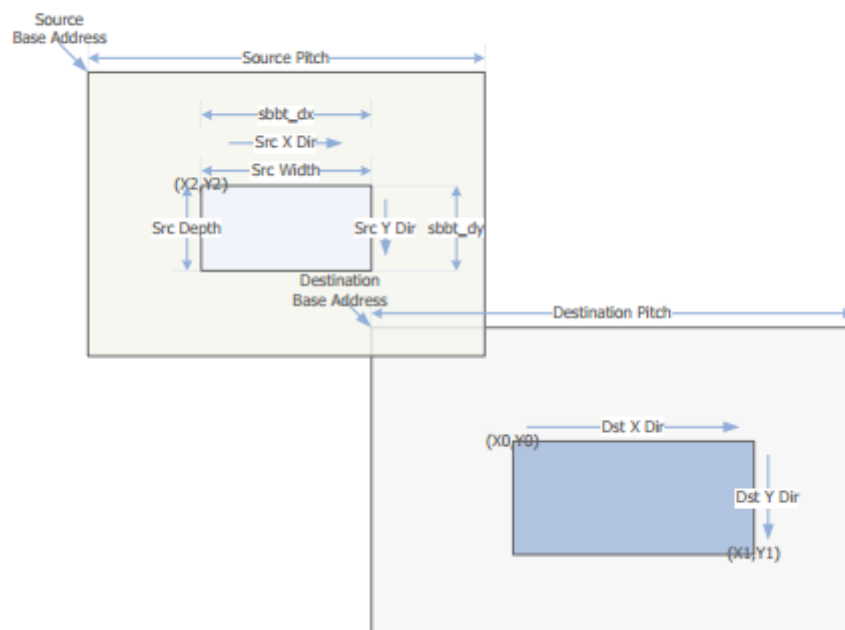


Figure 1-4

Other operations are not listed one by one, which is basically the same as the standard 2D acceleration.

1.2. Call flow diagram

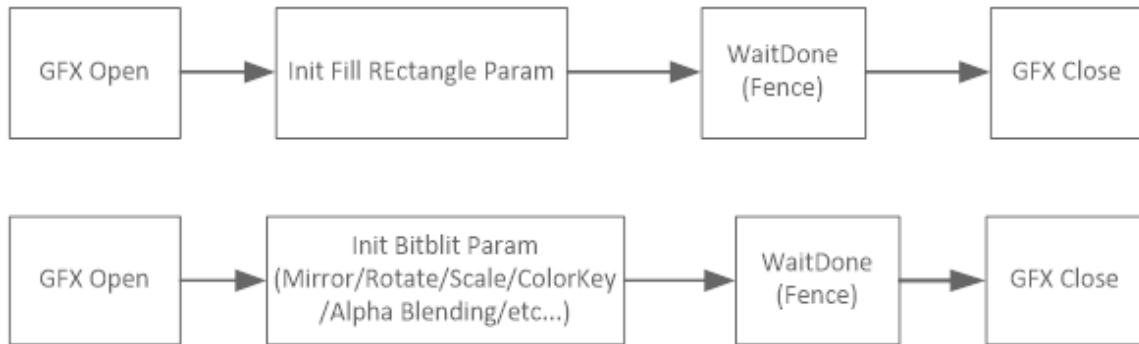


Figure 1-5

If users want to use GE to process video or UI data, they can use the following process

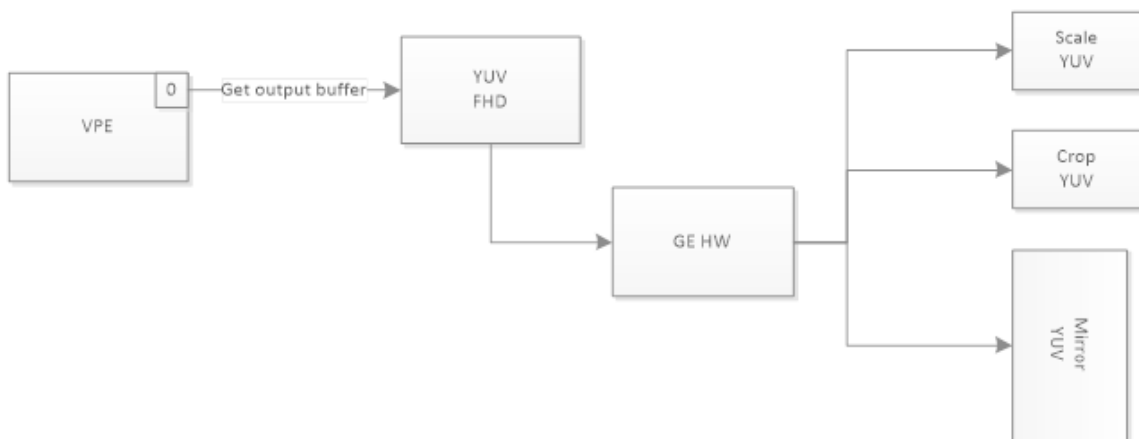


Figure 1-6

1.3. Keyword Description

- Fence

Corresponding to the state that a GFX operation needs to wait, it will block and wait, but there will also be a timeout mechanism inside to prevent fools.

- Colorkey

Key color, for example, when moving, you can remove a certain color in the bitmap, which can set this color as the key color

- Pitch

The number of bytes occupied by each line of pixels, $(\text{pixel bits}/8) * \text{Width}$, also corresponds to the commonly used Stride, LineLength

2. API Reference

2.1. Function Module API

API name	Features
MI_GFX_Open [#22-mi_gfx_open]	Open the GFX device
MI_GFX_Close [#23-mi_gfx_close]	Close the GFX device
MI_GFX_WaitAllDone [#24-mi_gfx_waitalldone]	Wait for all tasks of GFX to complete
MI_GFX_QuickFill [#25-mi_gfx_quickfill]	Add a quick fill action to a task
MI_GFX_GetAlphaThresholdValue [#26-mi_gfx_getalphathresholdvalue]	Get the alpha decision threshold
MI_GFX_SetAlphaThresholdValue [#27-mi_gfx_setalphathresholdvalue]	Set alpha decision threshold
MI_GFX_Bitblit [#28-mi_gfx_bitblit]	Add a move operation with additional functionality to the raster bitmap to the task
MI_GFX_SetPalette [#29-mi_gfx_setpalette]	设置索引颜色格式的调色板
MI_GFX_InitDev [#210-mi_gfx_initdev]	初始化GFX设备
MI_GFX_DeInitDev [#211-mi_gfx_deinitdev]	反初始化GFX设备
MI_GFX_DrawLine [#212-mi_gfx_drawline]	向任务中添加画线操作

2.2. MI_GFX_Open

- 功能

调用此接口打开GFX 设备并初始化硬件设备。

- 语法

```
MI_ S32 MI_GFX_Open(void);
```

- 返回值
 - MI_SUCCESS 打开设备成功。
 - 初始化失败，详情参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_gfx.h
 - 库文件：libmi_gfx.a/libmi_gfx.so
- 注意
 - 在进行 GFX相关操作前应该首先调用此接口，保证 GFX 设备处于打开状态。
 - GFX设备初始化一次后，再次调用将返回已初始化，可重复调用。
- 举例

```
1. MI_S32 s32Ret = 0;
2.
3. MI_SYS_Init();
4. /* open GFX device*/
5. s32Ret = MI_GFX_Open();
6. if (MI_SUCCESS != s32Ret)
7. {
8.     return -1;
9. }
10. /* close GFX device*/
11. MI_GFX_Close();
12. MI_SYS_Exit();
```

2.3. MI_GFX_Close

- 功能

调用此接口关闭 GFX设备。
- 语法

```
MI_ S32 MI_GFX_Close(void);
```

- 返回值
 - MI_SUCCESS 成功销毁GFX模块。
 - 销毁GFX模块失败，详情参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_gfx.h
 - 库文件：libmi_gfx.a/libmi_gfx.so
- 注意
 - 调用 [MI_GFX_Open](#) [#22-mi_gfx_open] 与 [MI_GFX_Close](#) [#23-mi_gfx_close] 的次数需要对应，可以重复调用，Open的次数和Close的次数一致时，GFX的api将不再能调用。

2.4. MI_GFX_WaitAllDone

- 功能

调用此接口等待 GFX 的所有任务完成或者等待指定的任务完成。
- 语法

```
MI_S32 MI_GFX_WaitAllDone(MI_BOOL bWaitAllDone, MI_penU16 u16TargetFence);
```

- 形参

参数名称	描述	输入/输出
bWaitAllDone	是否等待所有GFX任务完成	输入
u16TargetFence	等待指定的fence完成，在bWaitAllDone为FALSE时才有效	输入

- 返回值
 - MI_SUCCESS 等待的GFX任务都已经完成。
 - 等待GFX任务完成失败，详情参照[错误码](#) [#jump]。

- 依赖
 - 头文件：mi_gfx.h
 - 库文件：libmi_gfx.a/libmi_gfx.so
- 注意
 - 在调用此接口前应保证调用 [MI_GFX_Open](#) [#22-mi_gfx_open] 打开 GFX 设备。
 - 此接口为阻塞接口，会阻塞等待所有的 GFX 任务完成。

2.5. MI_GFX_QuickFill

- 功能

向目标位图pstDst中的目标区域pstDstRect，填入值为u32ColorVal的颜色，实现颜色填充的功能。

- 语法

```
MI_S32 MI_GFX_QuickFill(MI_GFX_Surface_t *pstDst, MI_GFX_Rect_t *pstDstRect, MI_U32 u32ColorVal, MI_U16 *pu16Fence);
```

- 形参

参数名称	描述	输入/输出
pstDst	目标位图	输入
pstDstRect	目标位图操作区域	输入
u32ColorVal	填充颜色值	输入
pu16Fence	返回的Fence指针	输出

- 返回值
 - MI_SUCCESS 成功添加QuickFill行为的任务到HW队列。
 - 填充色块失败，详情参照[错误码](#) [#jump]。

- 依赖
 - 头文件：mi_gfx.h：mi_gfx.h
 - 库文件：libmi_gfx.a/libmi_gfx.so
- 注意
 - 在调用此接口前应保证调用 [MI_GFX_Open](#) [#22-mi_gfx_open] 打开 GFX 设备。
 - 目标位图操作区域不能超出目标位图的大小。
 - 如果是目标位图的像素格式为RGB或ARGB，u32ColorVal从高位到低位，依次是A8：R8：G8：B8，如果目标位图的像素格式为YUYV422，则A8则无效，R对应Y，G对应U，B对应V。
- 举例

```

1. FILE *fp = NULL;
2. MI_PHY phyAddr;
3. void *pVirAddr;
4. MI_GFX_Surface_t stDst;
5. MI_GFX_Rect stDstRect;
6. MI_U32 u32ColorVal = 0xffffffff;
7. MI_U16 u16TargetFence;
8.
9. fp = fopen(SRC_FILE_NAME, "wb");
10.
11. ExecFunc(MI_SYS_Init(), MI_SUCCESS);
12. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0",
SRC_WIDTH*SRC_HEIGHT*2, &phyAddr), MI_SUCCESS);
13.
14. //MI_U32 phySrcAddr, phyDstAddr;
15. ExecFunc(MI_GFX_Open(), MI_SUCCESS);
16. ExecFunc(MI_SYS_Mmap(phyAddr, SRC_WIDTH*SRC_HEIGHT*2,
&pVirAddr, FALSE), MI_SUCCESS);
17. memset(pVirAddr, 0x22, SRC_WIDTH*SRC_HEIGHT*2);
18.
19. //fillrect
20. memset(&stDst, 0x0, sizeof(stDst));
21. stDst.eColorFmt = E_MI_GFX_FMT_ARGB1555;
22. stDst.u32Width = SRC_WIDTH;
23. stDst.u32Height = SRC_HEIGHT;
24. stDst.u32Stride = SRC_WIDTH * 2;
25. stDst.phyAddr = phyAddr;
26.
27. memset(&stDstRect, 0x0, sizeof(stDstRect));

```

```

28. stDstRect.s32Xpos = 100;
29. stDstRect.s32Ypos = 100;
30. stDstRect.u32Width = 100;
31. stDstRect.u32Height = 100;
32.
33. ExecFunc(MI_GFX_QuickFill(&stDst, &stDstRect, u32ColorVal,
    &u16TargetFence), MI_SUCCESS);
34. ExecFunc(MI_GFX_WaitAllDone(FALSE, u16TargetFence),
    MI_SUCCESS);
35. if (NULL != fp)
36. {
37.     fwrite(pVirAddr, 1, SRC_WIDTH*SRC_HEIGHT*2, fp);
38.     fclose(fp);
39.     fp = NULL;
40. }
41. ExecFunc(MI_SYS_Munmap(pVirAddr, SRC_WIDTH*SRC_HEIGHT*2),
    MI_SUCCESS);
42. ExecFunc(MI_SYS_MMA_Free(phyAddr), MI_SUCCESS);
43.
44. ExecFunc(MI_GFX_Close(), MI_SUCCESS);
45. ExecFunc(MI_SYS_Exit(), MI_SUCCESS);

```

2.6. MI_GFX_GetAlphaThresholdValue

- 功能

获取 alpha 判决阈值。

当源位图和目标位图做 **bitblit** 操作时，不管源位图和目标位图的像素格式是什么，硬件都会各生成一个像素格式为 **ARGB8888** 的中间位图，再将中间位图的运算结果输出到目标位图。当目标图片像素格式为 **ARGB1555** 时，若中间位图运算后某像素的 **alpha** 值小于此阈值，则输出到目标位图的该像素 **alpha** 位取0；大于或等于此阈值，该像素的 **alpha** 位取 1。

- 语法

```
MI_S32 MI_GFX_GetAlphaThresholdValue(MI_U8 *pu8ThresholdValue);
```

- 形参

参数名称	描述	输入/输出
pu8ThresholdValue	指向 alpha 判决阈值的指针	输出

- 返回值
 - MI_SUCCESS 成功获取到alpha阈值。
 - 获取alpha阈值失败，详情参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_gfx.h：mi_gfx.h
 - 库文件：libmi_gfx.a/libmi_gfx.so

2.7. MI_GFX_SetAlphaThresholdValue

- 功能

设置 alpha 判决阈值。

当源位图和目标位图做 bitblit 操作时，不管源位图和目标位图的像素格式是什么，硬件都会各生成一个像素格式为 ARGB8888 的中间位图，再将中间位图的运算结果输出到目标位图。当目标图片像素格式为 ARGB1555 时，若中间位图运算后某像素的 alpha 值小于此阈值，则输出到目标位图的该像素 alpha 位取0；大于或等于此阈值，该像素的 alpha 位取 1。

- 语法

```
MI_S32 MI_GFX_SetAlphaThresholdValue(MI_U8 u8ThresholdValue);
```

- 形参

数名称	描述	输入/输出
u8ThresholdValue	alpha 判决阈值	输入

- 返回值

- MI_SUCCESS 成功设置alpha阈值。
- 设置alpha阈值失败，详情参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_gfx.h：mi_gfx.h
 - 库文件：libmi_gfx.a/libmi_gfx.so

2.8. MI_GFX_BitBlit

- 功能

将源位图 (pstSrc) 与目标位图 (pstDst) 的指定区域 (pstSrcRect 、 pstDstRect) 进行运算，将运算后的位图拷贝到目标位图 (pstDst) 的指定区域 (pstDstRect) 中。

- 语法

```
MI_S32 MI_GFX_BitBlit(MI_GFX_Surface_t *pstSrc, MI_GFX_Rect_t
*pstSrcRect, MI_GFX_Surface_t *pstDst, MI_GFX_Rect_t
*pstDstRect, MI_GFX_Opt_t *pstOpt,
MI_U16 *pu16Fence);
```

- 形参

参数名称	描述	输入/输出
pstSrc	源位图。	输入
pstSrcRect	源位图操作区域。	输入
pstDst	目标位图。	输入
pstDstRect	目标位图操作区域。	输入
pstOpt	运算参数设置结构。	输入
pu16Fence	返回的Fence指针	输出

- 返回值
 - MI_SUCCESS 初始化成功。
 - 返回失败，详情参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_gfx.h：mi_gfx.h
 - 库文件：libmi_gfx.a/libmi_gfx.so
- 注意
 - 在调用此接口前应保证调用 [MI_GFX_Open](#) [#22-mi_gfx_open] 打开 GFX 设备。
 - 当源位图操作区域与目标位图操作区域尺寸不一致时，源区域按照目标区域比例进行缩放后，再与目标区域做运算。
 - clip 操作时，裁减区域必须与目标位图操作区域有公共交集，否则会返回错误。
 - MI_GFX_Opt_t 结构中存放有 GFX 运算功能的配置信息，如：是否作色键（colorkey）及 colorkey 的配置值；是否作区域裁减（clip 操作）及指定 clip 区域；是否镜像、是否进行 alpha 混合等信息。上述的操作可以同时使能。
- 举例

```

1. FILE *fp = NULL;
2. FILE *dstfp = NULL;
3. MI_PHY phyAddr, phyAddr2;
4. void *pVirAddr = NULL, *pVirAddr2 = NULL;
5. MI_GFX_Surface_t stSrc, stDst;
6. MI_GFX_Rect_t stSrcRect, stDstRect;
7. MI_U16 u16TargetFence;
8.
9. fp = fopen(SRC_FILE_NAME, "wb");
10. dstfp = fopen(DST_FILE_NAME, "wb");
11.
12. ExecFunc(MI_SYS_Init(), MI_SUCCESS);
13. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0",
SRC_WIDTH*SRC_HEIGHT*2, &phyAddr), MI_SUCCESS);
14. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0",
DST_WIDTH*360*2, &phyAddr2), MI_SUCCESS);
15.
16. //MI_U32 phySrcAddr, phyDstAddr;

```

```
17. ExecFunc(MI_GFX_Open(), MI_SUCCESS);
18. ExecFunc(MI_SYS_Mmap(phyAddr, SRC_WIDTH*SRC_HEIGHT*2,
&pVirAddr , FALSE), MI_SUCCESS);
19. memset(pVirAddr, 0x22, SRC_WIDTH*SRC_HEIGHT*2);
20.
21. ExecFunc(MI_SYS_Mmap(phyAddr2, DST_WIDTH*DST_HEIGHT*2,
&pVirAddr2 , FALSE), MI_SUCCESS);
22. memset(pVirAddr2, 0x0F, DST_WIDTH*DST_HEIGHT*2);
23.
24. //bitblit
25. memset(&stSrc, 0x0, sizeof(stSrc));
26. stSrc.eColorFmt = E_MI_GFX_FMT_ARGB1555;
27. stSrc.u32Width = SRC_WIDTH;
28. stSrc.u32Height = SRC_HEIGHT;
29. stSrc.u32Stride = SRC_WIDTH * 2;
30. stSrc.phyAddr = phyAddr;
31.
32. memset(&stSrcRect, 0x0, sizeof(stSrcRect));
33. stSrcRect.s32Xpos = 100;
34. stSrcRect.s32Ypos = 100;
35. stSrcRect.u32Width = 300;
36. stSrcRect.u32Height = 300;
37.
38. memset(&stDst, 0x0, sizeof(stDst));
39. stDst.eColorFmt = E_MI_GFX_FMT_ARGB1555;
40. stDst.u32Width = DST_WIDTH;
41. stDst.u32Height = DST_HEIGHT;
42. stDst.u32Stride = DST_WIDTH * 2;
43. stDst.phyAddr = phyAddr2;
44.
45. memset(&stDstRect, 0x0, sizeof(stDstRect));
46. stDstRect.s32Xpos = 200;
47. stDstRect.s32Ypos = 100;
48. stDstRect.u32Width = 200;
49. stDstRect.u32Height = 100;
50.
51. ExecFunc(MI_GFX_BitBlit(&stSrc, &stSrcRect, &stDst,
&stDstRect, NULL, &u16TargetFence), MI_SUCCESS);
52. ExecFunc(MI_GFX_WaitAllDone(FALSE, u16TargetFence),
MI_SUCCESS);
53. if (NULL != dstfp)
54. {
55.     fwrite(pVirAddr2, 1, DST_WIDTH*DST_HEIGHT*2, dstfp);
56.     fclose(dstfp);
57.     dstfp = NULL;
58. }
59.
60. ExecFunc(MI_SYS_Munmap(pVirAddr, SRC_WIDTH*SRC_HEIGHT*2),
MI_SUCCESS);
```

```
61. ExecFunc(MI_SYS_MMA_Free(phyAddr), MI_SUCCESS);
62.
63. ExecFunc(MI_SYS_Munmap(pVirAddr2, DST_WIDTH*DST_HEIGHT*2),
MI_SUCCESS);
64. ExecFunc(MI_SYS_MMA_Free(phyAddr2), MI_SUCCESS);
65.
66. ExecFunc(MI_GFX_Close(), MI_SUCCESS);
67. ExecFunc(MI_SYS_Exit(), MI_SUCCESS);
```

2.9. MI_GFX_SetPalette

- 功能

为索引颜色格式(I2/I4/I8)设置调色板

- 语法

```
MI_S32 MI_GFX_SetPalette(MI_GFX_ColorFmt_e eColorFmt,
MI_GFX_Palette_t *pstPalette);
```

- 形参

参数名称	描述	输入/输出
eColorFmt	调色板对应的颜色格式	输入
pstPalette	调色板数据	输入

- 返回值

- MI_SUCCESS 初始化成功。
- 返回失败，详情参照[错误码](#) [jump]。

- 依赖

- 头文件：mi_gfx.h：mi_gfx.h
- 库文件：libmi_gfx.a/libmi_gfx.so

- 注意

- 在调用此接口前应保证调用 [MI_GFX_Open](#) [#22-mi_gfx_open] 打开 GFX 设备。
- `pstPalette` 是一个容量为256的[MI_GFX_PaletteEntry_t](#) [#313-mi_gfx_paletteentry_t] 数组,每一个[MI_GFX_PaletteEntry_t](#) [#313-mi_gfx_paletteentry_t]代表一种颜色，对应数组中的下标代表了颜色的索引。
- 举例

```

1. MI_GFX_Palette_t myPal;
2. MI_GFX_PaletteEntry_t index_0;
3. MI_GFX_PaletteEntry_t index_1;
4. index_0.RGB.u8A = 0x40;
5. index_0.RGB.u8R = 0xFF;
6. index_0.RGB.u8G = 0;
7. index_0.RGB.u8B = 0;
8. index_1.RGB.u8A = 0x40;
9. index_1.RGB.u8R = 0;
10. index_1.RGB.u8G = 0xFF;
11. index_1.RGB.u8B = 0;
12.
13. myPal.aunPalette[0] = index_0;
14. myPal.aunPalette[1] = index_1;
15. myPal.u16PalStart = 0
16. myPal.u16PalEnd = 1;
17. MI_GFX_SetPalette(E_MI_GFX_FMT_I8, &myPal);

```

2.10. MI_GFX_InitDev

- 功能
初始化gfx设备。
- 语法

```
MI_S32 MI_GFX_InitDev(MI_GFX_InitParam_t *pstInitParam);
```

- 形参

参数名称	描述	输入/输出
pstInitParam	设备初始化参数	输入

- 返回值
 - MI_SUCCESS 成功。
 - 失败，详情参照错误码。
- 依赖
 - 头文件：mi_gfx.h
 - 库文件：libmi_gfx.a/libmi_gfx.so

2.11. MI_GFX_DeInitDev

- 功能
反初始gfx设备。
- 语法

```
MI_ S32 MI_GFX_DeInitDev (void);
```

- 返回值
 - MI_SUCCESS 成功。
 - 失败，详情参照错误码。
- 依赖
 - 头文件：mi_gfx.h
 - 库文件：libmi_gfx.a/libmi_gfx.so
- 注意
 - 此函数必须在初始化设备后调用，否则返回失败。
 - 如果本接口在app退出前没有调用，内部会自动反初始化设备。
- 举例

```
1. MI_GFX_Palette_t myPal;

2. MI_GFX_PaletteEntry_t index_0;

3. MI_GFX_PaletteEntry_t index_1;

4. index_0.RGB.u8A = 0x40;

5. index_0.RGB.u8R = 0xFF;

6. index_0.RGB.u8G = 0;
7. index_0.RGB.u8B = 0;

8. index_1.RGB.u8A = 0x40;

9. index_1.RGB.u8R = 0;

10. index_1.RGB.u8G = 0xFF;

11. index_1.RGB.u8B = 0;

12. myPal.aunPalette[0] = index_0;

13. myPal.aunPalette[1] = index_1;

14. myPal.u16PalStart = 0

15. myPal.u16PalEnd = 1;

16. MI_GFX_SetPalette(E_MI_GFX_FMT_I8, &myPal);
```

2.12. MI_GFX_DrawLine

- 功能

向任务中加入画线操作。

- 语法

```
MI_S32 MI_GFX_DrawLine(MI_GFX_Surface_t *pstDst, MI_GFX_Line_t
*pstLine, MI_U16 *pu16Fence);
```

- 形参

参数名称	描述	输入/输出
pstDst	目标位图	输入
pstLine	直线属性	输入
pu16Fence	返回Fence的指针	输出

- 返回值
 - MI_SUCCESS 成功。
 - 失败，详情参照错误码。
- 依赖
 - 头文件：mi_gfx.h
 - 库文件：libmi_gfx.a/libmi_gfx.so
- 注意
 - 在调用此接口前应保证调用 MI_GFX_Open 打开 GFX 设备。
 - 画线区域不能超出目标位图的大小。
 - 直线的终点不会被画出。
 - 如果是目标位图的像素格式为RGB或ARGB，
u32ColorFrom/u32ColorFrom从高位到低位，依次是A8：R8：G8：B8，
如果目标位图的像素格式为YUYV422，则A8则无效，R对应Y，G对应U，
B对应V。
- 举例

```
1. FILE *fp = NULL;
2. MI_GFX_Surface_t stDst;
3. MI_GFX_Line_t stLine;
4. MI_U16 u16TargetFence;
5. MI_PHY phyAddr;
6. void *VirAddr;
7.
8. ExecFunc(MI_SYS_Init(), MI_SUCCESS);
9. fp = fopen(SRC_FILE_NAME, "wb");
10. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0",
SRC_WIDTH*SRC_HEIGHT*2, &phyAddr), MI_SUCCESS);
```

```

11.
12. //MI_U32 phySrcAddr, phyDstAddr;
13. ExecFunc(MI_GFX_Open(), MI_SUCCESS);
14. ExecFunc(MI_SYS_Mmap(phyAddr, SRC_WIDTH*SRC_HEIGHT*2,
&pVirAddr , FALSE), MI_SUCCESS);
15. memset(pVirAddr, 0x22, SRC_WIDTH*SRC_HEIGHT*2);
16.
17. //draw line
18. memset(&stDst, 0x0, sizeof(stDst));
19. stDst.eColorFmt = E_MI_GFX_FMT_ARGB1555;
20. stDst.u32Width = SRC_WIDTH;
21. stDst.u32Height = SRC_HEIGHT;
22. stDst.u32Stride = SRC_WIDTH * 2;
23. stDst.phyAddr = phyAddr;
24.
25. memset(&stLine, 0x0, sizeof(stLine));
26. stLine.stPointFrom.s16x = 0;
27. stLine.stPointFrom.s16y = 0;
28. stLine.stPointTo.s16x = 100;
29. stLine.stPointTo.s16y = 100;
30. stLine.u16Width = 1;
31. stLine.bColorGradient = TRUE;
32. stLine.bColorFrom = 0xffff0000;
33. stLine.bColorTo = 0xffff00ff;
34.
35. ExecFunc(MI_GFX_DrawLine(&stDst, &stLine, &u16TargetFence),
MI_SUCCESS);
36. ExecFunc(MI_GFX_WaitAllDone(FALSE, u16TargetFence),
MI_SUCCESS);
37. if (NULL != fp)
38. {
39.     fwrite(pVirAddr, 1, SRC_WIDTH*SRC_HEIGHT*2, fp);
40.     fclose(fp);
41.     fp = NULL;
42. }
43. ExecFunc(MI_SYS_Munmap(pVirAddr, SRC_WIDTH*SRC_HEIGHT*2),
MI_SUCCESS);
44. ExecFunc(MI_SYS_MMA_Free(phyAddr), MI_SUCCESS);
45.
46. ExecFunc(MI_GFX_Close(), MI_SUCCESS);
47. ExecFunc(MI_SYS_Exit(), MI_SUCCESS);

```

3. GFX 数据类型

3.1. GFX数据类型说明

数据类型	说明
MI_GFX_ColorFmt_e [#32-mi_gfx_colorfmt_e]	定义像素颜色格式
MI_GFX_Surface_t [#33-mi_gfx_surface_t]	定义位图属性结构体
MI_GFX_Rect_t [#34-mi_gfx_rect_t]	定义操作区域属性结构体
MI_GFX_ColorKeyOp_e [#35-mi_gfx_colorkeyop_e]	定义Colorkey 操作模式
MI_GFX_ColorKeyValue_t [#36-mi_gfx_colorkeyvalue_t]	定义Colorkey 颜色属性结构体
MI_GFX_ColorKeyInfo_t [#37-mi_gfx_colorkeyinfo_t]	定义Colorkey 属性结构体
MI_GFX_Rotate_e [#38-mi_gfx_rotate_e]	定义图像旋转角度
MI_GFX_Mirror_e [#39-mi_gfx_mirror_e]	定义图像镜像模式
MI_GFX_DfbBldOp_e [#310-mi_gfx_dfbldop_e]	定义Alpha Blending 的模式
MI_Gfx_DfbBlendFlags_e [#311-mi_gfx_dfbblendflags_e]	定义Alpha Blending 的标志
MI_GFX_Opt_t [#312-mi_gfx_opt_t]	定义Bitblit 操作的可选属性结构体
MI_GFX_PaletteEntry_t [#313-mi_gfx_paletteentry_t]	定义调色板颜色结构体
MI_GFX_Palette_t [#314-mi_gfx_palette_t]	定义调色板结构体
MI_GFX_InitParam_t [#315-mi_gfx_initparam_t]	定义GFX设备初始化参数
MI_GFX_Point_t [#316-mi_gfx_point_t]	定义点属性结构体
MI_GFX_Line_t [#317-mi_gfx_line_t]	定义直线属性结构体

3.2. MI_GFX_ColorFmt_e

- 说明

定义像素颜色格式。

- 定义

```
typedef enum
{
    E_MI_GFX_FMT_I1 = 0, /* ColorFormat */
    E_MI_GFX_FMT_I2,
    E_MI_GFX_FMT_I4,
    E_MI_GFX_FMT_I8,
    E_MI_GFX_FMT_FABAFGBG2266,
    E_MI_GFX_FMT_1ABFGBG12355,
    E_MI_GFX_FMT_RGB565,
    E_MI_GFX_FMT_ARGB1555,
    E_MI_GFX_FMT_ARGB4444,
    E_MI_GFX_FMT_ARGB1555_DST,
    E_MI_GFX_FMT_YUV422,
    E_MI_GFX_FMT_ARGB8888,
    E_MI_GFX_FMT_RGBA5551,
    E_MI_GFX_FMT_RGBA4444,
    E_MI_GFX_FMT_ABGR8888,
    E_MI_GFX_FMT_BGRA5551,
    E_MI_GFX_FMT_ABGR1555,
    E_MI_GFX_FMT_ABGR4444,
    E_MI_GFX_FMT_BGRA4444,
```

```

    E_MI_GFX_FMT_BGR565,

    E_MI_GFX_FMT_RGBA8888,

    E_MI_GFX_FMT_BGRA8888,

    E_MI_GFX_FMT_MAX

} MI_GFX_ColorFmt_e;

```

- 注意事项

对应surface支持的color format，在bitblit操作时，对应的SrcSurface、DstSurface的颜色格式。

- 相关数据类型及接口

[MI_GFX_ColorKeyInfo_t](#) [#37-mi_gfx_colorkeyinfo_t]

[MI_GFX_Surface_t](#) [#33-mi_gfx_surface_t]

[MI_GFX_SetPalette](#) [#29-mi_gfx_setpalette]

3.3. MI_GFX_Surface_t

- 说明

定义位图属性结构体。

- 定义

```

typedef struct MI_GFX_Surface_s
{
    MI_PHY phyAddr;

    MI_GFX_ColorFmt_e eColorFmt;

    MI_U32 u32Width;

    MI_U32 u32Height;

    MI_U32 u32Stride;

} MI_GFX_Surface_t;

```


- 成员

成员	描述
u32PhyAddr	位图对应内存的物理地址。
eColorFmt	位图像素颜色格式。
u32Height	位图高度。
u32Width	位图宽度。
u32Stride	位图跨度。

- 注意事项

- 当位图的像素格式大小大于或等于 1Byte 时，位图对应内存的物理地址和 Stride 必须按照像素格式大小对齐；当位图的像素格式大小不足 1Byte 时，位图对应内存的物理地址和 Stride 需要按照 1Byte 对齐。
- YUYV422 格式的位图的水平起始位置和宽度必须为偶数。

- 相关数据类型及接口

[MI_GFX_QuickFill](#) [#25-mi_gfx_quickfill]

[MI_GFX_DrawLine](#) [#212-mi_gfx_drawline]

[MI_GFX_Bitblit](#) [#28-mi_gfx_bitblit]

3.4. MI_GFX_Rect_t

- 说明

定义操作区域结构体。

- 定义

```
typedef struct MI_GFX_Rect_s
{
```

```
MI_S32 s32Xpos;

MI_S32 s32Ypos;

MI_U32 u32Width;

MI_U32 u32Height;

} MI_GFX_Rect_t;
```

- 成员

成员	描述
s32Xpos	操作区域的起始横坐标，以像素数为单位。有效范围[0, 位图宽度)
s32Ypos	操作区域的起始纵坐标，以像素数为单位。有效范围[0, 位图高度)
u32Width	操作区域的宽度，以像素数为单位。
u32Height	操作区域的高度，以像素数为单位。

- 注意事项

操作区域必须在位图范围内。

- 相关数据类型及接口

[MI_GFX_Opt_t](#) [#312-mi_gfx_opt_t]

[MI_GFX_QuickFill](#) [#25-mi_gfx_quickfill]

[MI_GFX_Bitblit](#) [#28-mi_gfx_bitblit]

3.5. MI_GFX_ColorKeyOp_e

- 说明

定义 colorkey 操作模式。

- 定义

```
typedef enum
{
    E_MI_GFX_RGB_OP_EQUAL = 0,
    E_MI_GFX_RGB_OP_NOT_EQUAL,
    E_MI_GFX_ALPHA_OP_EQUAL,
    E_MI_GFX_ALPHA_OP_NOT_EQUAL,
    E_MI_GFX_ARGB_OP_EQUAL,
    E_MI_GFX_ARGB_OP_NOT_EQUAL,
    E_MI_GFX_CKEY_OP_BUTT,
} MI_GFX_ColorKey_e;
```

- 成员

成员	描述
E_MI_GFX_RGB_OP_EQUAL	RGB相等做colorkey 操作
E_MI_GFX_RGB_OP_NOT_EQUAL	RGB不相等做colorkey 操作
E_MI_GFX_ALPHA_OP_EQUAL	ALPHA相等做colorkey操作
E_MI_GFX_ALPHA_OP_NOT_EQUAL	ALPHA不相等做colorkey操作
E_MI_GFX_ARGB_OP_EQUAL	ARGB相等做colorkey 操作
E_MI_GFX_ARGB_OP_NOT_EQUAL	ARGB不相等做colorkey 操作
E_MI_GFX_CKEY_OP_BUTT	无效的colorkey 操作

- 相关数据类型及接口

[MI_GFX_ColorKeyInfo_t](#) [#37-mi_gfx_colorkeyinfo_t]

3.6. MI_GFX_ColorKeyValue_t

- 说明

定义colorkey的颜色结构体。

- 定义

```
typedef struct MI_GFX_ColorKey_s
{
    MI_U32 u32ColorStart;

    MI_U32 u32ColorEnd;

} MI_GFX_ColorKeyValue_t;
```

- 成员

成员	描述
u32ColorStart	ColorKey颜色范围起始值
u32ColorEnd	ColorKey颜色范围结束值

- 注意事项

当仅针对一个color值做colorkey操作时，将 u32ColorStart = u32ColorEnd = colorVal。

- 相关数据类型及接口

[MI_GFX_ColorKeyInfo_t](#) [#37-mi_gfx_colorkeyinfo_t]

3.7. MI_GFX_ColorKeyInfo_t

- 说明

定义 colorkey 属性结构体。

- 定义

```
typedef struct MI_GFX_ColorKeyInfo_s
{
    MI_BOOL bEnColorKey;

    MI_GFX_ColorKeyOp_e eKeyOp;

    MI_GFX_ColorFmt_e eKeyFmt;

    MI_GFX_ColorKeyValue_t stCKeyVal;
} MI_GFX_ColorKeyInfo_t;
```

- 成员

成员	描述
bEnColorKey	是否使能ColorKey操作
eKeyOp	ColorKey操作模式
stCKeyVal	ColorKey颜色范围
eKeyFmt	ColorKey的颜色格式

- 相关数据类型及接口

[MI_GFX_Opt_t](#) [#312-mi_gfx_opt_t]

3.8. MI_GFX_Rotate_e

- 说明

定义图像旋转角度。

- 定义

```
typedef enum
{
```

```

    E_MI_GFX_ROTATE_0 = 0,

    E_MI_GFX_ROTATE_90,

    E_MI_GFX_ROTATE_180,

    E_MI_GFX_ROTATE_270

} MI_GFX_Rotate_e;

```

- 相关数据类型及接口

[MI_GFX_Opt_t](#) [#312-mi_gfx_opt_t]

3.9. MI_GFX_Mirror_e

- 说明

定义图像镜像模式。

- 定义

```

typedef enum
{

    E_MI_GFX_MIRROR_NONE = 0,

    E_MI_GFX_MIRROR_HORIZONTAL,

    E_MI_GFX_MIRROR_VERTICAL,

    E_MI_GFX_MIRROR_BOTH,

    E_MI_GFX_MIRROR_MAX

} MI_GFX_Mirror_e;

```

- 成员

成员	描述
E_MI_GFX_MIRROR_NONE	输出图像不进行镜像操作
E_MI_GFX_MIRROR_HORIZONTAL	输出图像水平镜像
E_MI_GFX_MIRROR_VERTICAL	输出图像垂直镜像
E_MI_GFX_MIRROR_BOTH	输出图像水平+垂直镜像
E_MI_GFX_MIRROR_MAX	无效的镜像类型

- 相关数据类型及接口

[MI_GFX_Opt_t](#) [#312-mi_gfx_opt_t]

3.10. MI_GFX_DfbBldOp_e

- 说明
定义 Alpha Blending 的模式。
- 定义

```
typedef enum
{
    E_MI_GFX_DFB_BLD_ZERO = 0,
    E_MI_GFX_DFB_BLD_ONE,
    E_MI_GFX_DFB_BLD_SRCCOLOR,
    E_MI_GFX_DFB_BLD_INVSRCOLOR,
    E_MI_GFX_DFB_BLD_SRCALPHA,
    E_MI_GFX_DFB_BLD_INVSRCALPHA,
    E_MI_GFX_DFB_BLD_DESTALPHA,
```

```
E_MI_GFX_DFB_BLD_INVDESTALPHA,  
  
E_MI_GFX_DFB_BLD_DESTCOLOR,  
  
E_MI_GFX_DFB_BLD_INVDESTCOLOR,  
  
E_MI_GFX_DFB_BLD_SRCALPHASAT,  
  
E_MI_GFX_DFB_BLD_NONE,  
  
E_MI_GFX_DFB_BLD_MAX,  
  
} MI_GFX_DfbBldOp_e;
```

- 成员

成员	描述
E_MI_GFX_DFB_BLD_OP_ZERO	$\text{argb} *= 0.0$ 同directfb DSBF_ZERO
E_MI_GFX_DFB_BLD_OP_ONE	$\text{argb} *= 1.0$ 同directfb DSBF_ONE
E_MI_GFX_DFB_BLD_OP_SRCOLOR	$\text{argb} *= \text{Sargb}$ 同directfb DSBF_SRCOLOR
E_MI_GFX_DFB_BLD_OP_INVSRCOLOR	$\text{argb} *= 1.0 - \text{Sargb}$ 同directfb DSBF_INVSRCOLOR
E_MI_GFX_DFB_BLD_OP_SRCALPHA	$\text{argb} *= \text{Saaaa}$ 同directfb DSBF_SRCALPHA
E_MI_GFX_DFB_BLD_OP_INVSRCALPHA	$\text{argb} *= 1.0 - \text{Saaaa}$ 同directfb DSBF_INVSRCALPHA
E_MI_GFX_DFB_BLD_OP_DESTCOLOR	$\text{argb} *= \text{Dargb}$ 同directfb DSBF_DESTCOLOR
E_MI_GFX_DFB_BLD_OP_INVDESTCOLOR	$\text{argb} *= 1.0 - \text{Dargb}$ 同directfb DSBF_INVDESTCOLOR
E_MI_GFX_DFB_BLD_OP_DESTALPHA	$\text{argb} *= \text{Daaaa}$ 同directfb DSBF_DESTALPHA
E_MI_GFX_DFB_BLD_OP_INVDESTALPHA	$\text{argb} *= 1.0 - \text{Daaaa}$ 同directfb DSBF_INVDESTALPHA
E_MI_GFX_DFB_BLD_OP_SRCALPHASAT	$\text{rgb} *= \min(\text{Sa}, 1 - \text{Da})$ 同directfb DSBF_SRCALPHASAT
E_MI_GFX_BLD_MAX	无效的 alpha 混合模式

- 注意事项

在源位图和目标位图作叠加运算时，可以分别设置 Src 通道和 Dst 通道的叠加模式。现在支持 11 种叠加模式，用 [MI_GFX_DfbBldOp_e](#) [#310-

mi_gfx_dfbldop_e] 设置模式。

- 相关数据类型及接口

[MI_GFX_Opt_t](#) [#312-mi_gfx_opt_t]

3.11. MI_Gfx_DfbBlendFlags_e

- 说明

定义Alpha Blending 的标志

- 定义

```
typedef enum
{
    E_MI_GFX_DFB_BLEND_NOFX = 0x00000000,
    E_MI_GFX_DFB_BLEND_COLORALPHA = 0x00000001,
    E_MI_GFX_DFB_BLEND_ALPHACHANNEL = 0x00000002,
    E_MI_GFX_DFB_BLEND_COLORIZE = 0x00000004,
    E_MI_GFX_DFB_BLEND_SRC_PREMULTIPLY = 0x00000008,
    E_MI_GFX_DFB_BLEND_SRC_PREMULTICOLOR = 0x00000010,
    E_MI_GFX_DFB_BLEND_DST_PREMULTIPLY = 0x00000020,
    E_MI_GFX_DFB_BLEND_XOR = 0x00000040,
    E_MI_GFX_DFB_BLEND_DEMULTIPLY = 0x00000080,
    E_MI_GFX_DFB_BLEND_SRC_COLORKEY = 0x00000100,
    E_MI_GFX_DFB_BLEND_DST_COLORKEY = 0x00000200,
    E_MI_GFX_DFB_BLEND_MAX = 0x3FF
} MI_Gfx_DfbBlendFlags_e;
```

- 成员

成员	描述
E_MI_GFX_DFB_BLEND_NOFX	没有任何BLD操作
E_MI_GFX_DFB_BLEND_COLORALPHA	src alpha会根据const color value运算 $\text{src.a} = \text{const color.a};$ 同directfb DSBLIT_BLEND_COLORALPHA
E_MI_GFX_DFB_BLEND_ALPHACHANNEL	src alpha会根据const color value运算 $\text{src.a} = \text{src.a} * \text{const color.a};$ 同directfb DSBLIT_BLEND_ALPHACHANNEL
E_MI_GFX_DFB_BLEND_COLORIZE	将const color作为全局颜色，调整源颜色 同directfb DSBLIT_COLORIZE
E_MI_GFX_DFB_BLEND_SRC_PREMULTIPLY	使用源alpha预乘源颜色 $\text{src.r} = \text{src.r} * \text{src.a};$ 等 同directfb DSBLIT_SRC_PREMULTIPLY
E_MI_GFX_DFB_BLEND_SRC_PREMULTCOLOR	使用const alpha预乘源颜色 $\text{src.r} = \text{src.r} * \text{const.a};$ 等 同directfb DSBLIT_SRC_PREMULTCOLOR
E_MI_GFX_DFB_BLEND_DST_PREMULTIPLY	使用目的alpha预乘目的颜色 $\text{dst.r} = \text{dst.r} * \text{dst.a};$ 等 同directfb DSBLIT_DST_PREMULTIPLY
E_MI_GFX_DFB_BLEND_XOR	调节和预乘alpha $\text{src.a} ^= \text{dst.a};$ 同directfb DSBLIT_XOR
E_MI_GFX_DFB_BLEND_DEMULTIPLY	预乘到非预乘的转换，最好不要使用这样的操作 $\text{x.r} = ((\text{int})\text{x.r} << 8) / ((\text{int})\text{x.a} + 1);$ 等 同directfb DSBLIT_DEMULTIPLY
E_MI_GFX_DFB_BLEND_SRC_COLORKEY	如果src的颜色等于const color，则会被key掉

E_MI_GFX_DFB_BLEND_DST_COLORKEY	如果dst的颜色等于const color，则会被key掉
E_MI_GFX_DFB_BLEND_MAX	上面所有的action都会被使能

- 相关数据类型及接口

[MI_GFX_Opt_t](#) [#312-mi_gfx_opt_t]

3.12. MI_GFX_Opt_t

- 说明
定义Bitblit 操作的可选属性结构体
- 定义

```
typedef struct MI_GFX_Opt_s
{
    MI_GFX_Rect_t stClipRect;

    MI_GFX_ColorKeyInfo_t stSrcColorKeyInfo;

    MI_GFX_ColorKeyInfo_t stDstColorKeyInfo;

    MI_GFX_DfbBldOp_e eSrcDfbBldOp;

    MI_GFX_DfbBldOp_e eDstDfbBldOp;

    MI_GFX_Mirror_e eMirror;

    MI_GFX_Rotate_e eRotate;

    MI_Gfx_DfbBlendFlags_e eDFBBlendFlag;

    MI_U32 u32GlobalSrcConstColor;

    MI_U32 u32GlobalDstConstColor;
} MI_GFX_Opt_t;
```

- 成员

成员	描述
stSrcColorKeyInfo	Src surface colorkey 操作
stDstColorKeyInfo	Dst surface colorkey操作
stClipRect	clip 区域
eMirror	图像镜像类型
eSrcDfbBldOp	Src Surface BLEND操作的模式
eDstDfbBldOp	Dst Surface BLEND操作的模式
eRotate	图像旋转角度，0、90、180、270度旋转
eDFBBlendFlag	Alpha Blending的标志，如XOR、AlphaBlending、Colorize等操作
u32GlobalSrcConstColor	针对eDFBBlendFlag 操作对应的const参数，eDFBBlendFlag 操作有对应的mutli等等
u32GlobalDstConstColor	针对eDFBBlendFlag 操作对应的const参数，eDFBBlendFlag 操作有对应的mutli等等

- 注意事项

目前仅有u32GlobalSrcConstColor有效。

- 相关数据类型及接口

[MI_GFX_Bitblit](#) [#28-mi_gfx_bitblit]

3.13. MI_GFX_PaletteEntry_t

- 说明

定义调色板颜色结构体。

- 定义

```
typedef union
{
    /// ARGB8888 byte order

    struct
    {
        MI_U8 u8A;

        MI_U8 u8R;

        MI_U8 u8G;

        MI_U8 u8B;

    } RGB;

    // u8Data[0] = u8A

    // u8Data[1] = u8R

    // u8Data[2] = u8G

    // u8Data[3] = u8B

    MI_U8 u8Data[4];

} MI_GFX_PaletteEntry_t;
```

- 成员

成员	描述
RGB	索引颜色对应的RGB颜色的分量结构体表示
u8Data	索引颜色对应的RGB颜色的字节序表示

- 注意事项

注意定义中注释对颜色定义的描述，以便正确填写调色板数据。

- 相关数据类型及结果

[MI_GFX_Palette_t](#) [#314-mi_gfx_palette_t]

3.14. MI_GFX_Palette_t

- 说明

定义调色板结构体。

- 定义

```
typedef struct MI_GFX_Palette_s
{
    MI_GFX_PaletteEntry_t aunPalette[256];

    MI_U16 u16PalStart;

    MI_U16 u16PalEnd;
}MI_GFX_Palette_t;
```

- 成员

成员	描述
aunPalette	调色板索引对应的RGB颜色数组
u16PalStart	需要配置RGB颜色的索引集起始索引
u16PalEnd	需要配置RGB颜色的索引集结束索引

- 相关数据类型及结果

[MI_GFX_PaletteEntry_t](#) [#313-mi_gfx_paletteentry_t]

[MI_GFX_SetPalette](#) [#29-mi_gfx_setpalette]

3.15. MI_GFX_InitParam_t

- 说明

定义GFX设备初始化参数。

- 定义

```
typedef struct MI_GFX_InitParam_s
{
    MI_U32 u32DevId;

    MI_U8 *u8Data;
} MI_GFX_InitParam_t;
```

- 成员

成员名称	描述
u32DevId	设备ID
u8Data	数据指针buffer

- 相关数据类型及接口

[MI_GFX_InitDev](#) [#210-mi_gfx_initdev]

3.16. MI_GFX_Point_t

- 说明

定义点属性结构体。

- 定义

```
typedef struct MI_GFX_Point_s
{
```



```
MI_S16 s16x;  
  
MI_S16 s16y;  
  
} MI_GFX_Point_t;
```

- 成员

成员名称	描述
s16x	X坐标
S16y	Y坐标

- 相关数据类型及接口

[MI_GFX_Line_t](#) [#317-mi_gfx_line_t]

3.17. MI_GFX_Line_t

- 说明

定义直线属性结构体。

- 定义

```
typedef struct MI_GFX_Line_s  
{  
  
    MI_GFX_Point_t stPointFrom;  
  
    MI_GFX_Point_t srPointTo;  
  
    MI_U16 u16Width;  
  
    MI_BOOL bColorGradient;  
  
    MI_U32 u32ColorFrom;  
  
    MI_U32 u32ColorTo;  
  
} MI_GFX_Line_t;
```

- 成员

成员名称	描述
stPointFrom	直线的起点
srPointTo	直线的终点
u16Width	直线宽度
bColorGradient	是否颜色渐变
u32ColorFrom	渐变起始颜色
u32ColorTo	渐变结束颜色

- 相关数据类型及接口

[ML_GFX_DrawLine](#) [#212-mi_gfx_drawline]

4. 错误码

GFX API 错误码如下所示

错误代码	宏定义	描述
0xA00D2003	MI_ERR_GFX_INVALID_PARAM	非法参数
0xA00D2002	MI_ERR_GFX_DEV_BUSY	GFX设备忙
0xA00D2220	MI_ERR_GFX_NOT_INIT	设备未初始化
0xA00D2221	MI_ERR_GFX_DRV_NOT_SUPPORT	不支持的操作类型
0xA00D2222	MI_ERR_GFX_DRV_FAIL_FORMAT	不支持的格式类型
0xA00D2223	MI_ERR_GFX_NON_ALIGN_ADDRESS	地址未对齐
0xA00D2224	MI_ERR_GFX_NON_ALIGN_PITCH	Pitch对齐错误
0xA00D2225	MI_ERR_GFX_DRV_FAIL_OVERLAP	Option内存有叠加区域
0xA00D2226	MI_ERR_GFX_DRV_FAIL_STRETCH	缩放失败
0xA00D2228	MI_ERR_GFX_DRV_FAIL_LOCKED	设备被其他使用者锁住
0xA00D2229	MI_ERR_GFX_DRV_FAIL_BLTADDR	Bitblit起始地址错误

5. PROCFS介绍

5.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_gfx/mi_gfx0
```

```

-----Gfx Dev Attr-----
module init count:  2
Device id:  0
ARGB1555 alpha threshold:  1
ARGB1555 To ARGB8888 alpha value:  255
Scaling coefficient:  500 / 1000

Device id:  1
ARGB1555 alpha threshold:  1
ARGB1555 To ARGB8888 alpha value:  255
Scaling coefficient:  500 / 1000
-----

```

- 调试信息分析

记录当前GFX的使用状况以及device属性可以动态地获取到这些信息，方便调试和测试。

- 参数说明

参数		描述
device info	module init count	设备被 Open 的次数，由于 GFX 会有多个模块进行调用，并且不能确定是哪一个 module 会先调用 GFX 的初始化，所以设备允许多次 Open，会记录 Open 次数来在 Close 的时候做对应的计数判断是否需要真的 Close Device。
	Device id	GFX的设备ID
	ARGB1555 alpha threshold	When ARGB8888 is converted to ARGB1555, the alpha determination threshold inside GFX, if it is greater than N, alpha is 1, otherwise alpha is 0.
	ARGB1555 To ARGB8888 alpha value	When converting ARGB1555 to ARGB8888, GFX converts the alpha of the pixel whose ARGB1555 alpha bit is 1 to ARGB1555 To ARGB8888 alpha value.
	Scaling coefficient	Image geometric center offset